codigo.py
PY 15.2KB

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Found 10 web pages

Com base na análise passo a passo, **este arquivo é altamente suspeito e apresenta características típicas de malware (provavelmente um carregador ou backdoor)**. Ele não é um programa benigno.

A seguir, a análise dos pontos-chave:

1. Verificação de Localização Geográfica (Evitar a Rússia)

A função `_isRussianSystem()` no código verifica extensivamente se o sistema está localizado na Rússia, incluindo:

- Detecção de variáveis de ambiente relacionadas ao russo (`LANG` , `LC_ALL` , `USERNAME`).
- Verificação do fuso horário do sistema (como Moscow, MSK) e da faixa de offset UTC.
- Se for detectado um sistema russo, o programa **para diretamente a execução** (`return`).

Análise: Este é um comportamento clássico de malware. Muitos malwares (como Amadey, DarkGate) evitam intencionalmente infectar sistemas na Rússia ou países da CEI para evitar a aplicação da lei local ou porque os autores do malware estão nessas regiões.

2. Cadeia de Comando C2 Baseada em Blockchain (Solana)

O código não obtém endereços C2 (servidor de comando e controle) de forma tradicional, mas sim:

- Chama o método `getSignaturesForAddress` da blockchain Solana. [1] [9]
- Obtém o campo `memo` das transações e o analisa como JSON.

- Extrai o campo `link` desse JSON e realiza decodificação Base64 para obter o endereço de download real.

Análise: Usar blockchain como canal oculto de C2 é uma técnica avançada de evasão. Como os dados da blockchain são públicos e imutáveis, os atacantes podem atualizar dinamicamente o endereço do servidor malicioso sem serem bloqueados por mecanismos tradicionais de bloqueio de domínios/IP.

3. Download e Execução de Carga Maliciosa Ofuscada

Após obter os dados, o código:

- Baixa e extrai o ambiente Node.js (comportamento em `download_nodejs`).
- Gera um script JavaScript `i.js`, cujo conteúdo principal é `eval(atob('...'))`.
- Executa o script em segundo plano através do Node.js, ocultando a janela (`CREATE_NO_WINDOW`).

Análise: Este é um método típico de execução de cargas maliciosas. `eval(atob(...))` é usado para decodificar e executar código ofuscado em tempo de execução, dificultando a detecção estática. O ambiente Node.js é usado para fornecer funcionalidades completas de sistema (como rede, sistema de arquivos), e a execução em segundo plano visa ocultar a presença.

4. Persistência e Controle de Tempo

- O código verifica e atualiza o arquivo `init.json` no diretório inicial do usuário, registrando timestamps para controlar se deve executar novamente em até 2 dias.
- O código contém lógica de espera (`await asyncio.sleep(10)`), possivelmente para evitar sandboxes ou ambientes de análise.

Análise: Isso indica que ele foi projetado para execução persistente ou execução periódica, em vez de ser uma ferramenta de uso único.

Resumo

A combinação de "**evitar a Rússia + obter C2 via blockchain + execução dinâmica de código Node.js ofuscado**" constitui uma cadeia de ataque completa. Seu objetivo é executar código malicioso controlado remotamente na máquina da vítima, ao mesmo tempo que utiliza múltiplas técnicas para evitar detecção e análise.

Recomendação: Não execute este arquivo em nenhuma circunstância. Se ele já foi executado, recomenda-se verificar imediatamente os processos em segundo plano do sistema (especialmente processos `node.exe`), conexões de rede e entradas de inicialização.

10 web pages

Qual é a categoria principal deste malware?

Found 14 web pages

Com base nas características do código (C2 via blockchain Solana, download e execução de Node.js, verificação de sistema russo), a categoria principal deste malware é um **carregador (Loader)** ou **dropper (liberador)**, e pertence à linhagem de malware **GlassWorm**.

🧬 Evidências de Identificação

As técnicas centrais do código são altamente consistentes com o malware **GlassWorm** documentado publicamente:

- **Canal C2 idêntico**: O código utiliza o método `getSignaturesForAddress` para consultar o campo `memo` da blockchain Solana e obter instruções, sendo este o mecanismo de comando e controle típico do GlassWorm. [1] [12]
- **Comportamento de execução idêntico**: O código baixa o ambiente Node.js e executa código JavaScript ofuscado através dele, correspondendo exatamente ao comportamento do GlassWorm (baixar runtime Node.js e executar infostealer). [4] [9]
- **Detalhes coincidentes**: O código gera o arquivo `i.js`, e relatórios de segurança também mencionam a necessidade de verificar arquivos `i.js` suspeitos. Além disso, o código utiliza `init.json` para controle de persistência, o que também corresponde às características do GlassWorm. [4] [9]

📌 Posicionamento Funcional

Embora o código contenha lógica para baixar e executar cargas adicionais, ele próprio não é o "ladrão de informações" (Infostealer) final. Sua função principal é **estabelecer comunicação com o servidor de controle, baixar o ambiente de execução e carregar a**

carga maliciosa final. Portanto, na classificação de malware, ele desempenha o papel de **carregador/primeira fase (Stage 1)**.

Resumindo, trata-se de um **carregador GlassWorm** que utiliza a blockchain Solana como canal C2 e tem como alvo principal o ambiente de desenvolvedores.

14 web pages